Structured Teamwork in Multi-Agent PPO: A Seven-Game Study of Coordination Primitives on Knights-Archers-Zombies

Ishan Biswas

Elizabeth Coquillette

Nishant Suresh[1]

[1]Advanced Deep Learning, CS7180 2025–2026

April 18, 2026

Abstract

We study cooperative multi-agent reinforcement learning on PettingZoo's Knights-Archers-Zombies (KAZ) environment through a controlled seven-game ablation that isolates the contribution of four coordination primitives: (i) ammo and stamina discipline, (ii) role assignment (forward knight vs. end-zone patrol knight), (iii) hard target locks with cross-role skip rules, and (iv) a pragmatic archer override that violates locks when a zombie is projected to cross the end line before a knight can intercept. Three games (**G0**–**G1b**) are evaluated with scripted heuristics to establish a ceiling and two constrained floors; four games (**G2**–**G5**) are trained with shared-parameter MAPPO using entity self-attention (**G2**) or attention + GRU (**G3**–**G5**) with transfer-learning between adjacent games. All games share identical episode length, spawn pressure, and per-episode evaluation seeds so that score differences isolate the primitive. Our main finding is that the learned progression **G2** → **G5** is monotonic in both stochastic and deterministic regimes, and the peak learned policy (**G5**) beats the unrestricted heuristic ceiling (**G0**) by +29%. The pragmatic override (**G5**) simultaneously maximises score and minimises failures, evidence that it performs a genuine offence / defence trade-off.

1. Introduction

Cooperative MARL on shared-objective games such as KAZ traditionally combines reward shaping with architectural upgrades (recurrence, attention, centralised critics). This paper takes a complementary route. Rather than asking which reward mix is best, we ask: *how much cooperative performance can be recovered by building structure into the environment itself?* We define four progressively strict coordination primitives, fix every other variable (episode length, spawn pressure, architecture family, evaluation seeds), and measure the marginal contribution of each primitive.

Contributions.

1. A seven-game ablation (**G0**–**G5**) that isolates ammo discipline, role assignment, target locks, and pragmatic override under fixed spawn pressure and evaluation seeds.
2. A seed-paired evaluation protocol in which episode i uses the same random seed across every game, so zombie spawn patterns are identical and score deltas are directly attributable to primitives.

3. Quantitative evidence that the learned progression is monotonic on both stochastic and deterministic evaluation: stochastic $1.80 \to 4.10 \to 7.40 \to 11.20$; deterministic $-1.90 \to 0.60 \to 0.20 \to 2.90$.
4. Identification of the pragmatic override as the single largest step in the progression (**G4** $\to$ **G5**: $+3.80$ stochastic, $+2.70$ deterministic), and the only primitive that simultaneously raises kills and lowers failures.

2. Background and Related Work

MAPPO. Yu et al. [10] showed that Proximal Policy Optimisation [7] with shared parameters is a competitive baseline for cooperative MARL without centralised critics or inter-agent communication. We adopt the shared-parameter formulation.

Reward shaping. Ng et al. [6] prove that potential-based shaping preserves the optimal policy; non-potential shaping can distort it. We therefore use minimal shaping (role and lock bonuses at 10^{-3}–10^{-2}) and move coordination into the wrapper, letting us study structure-vs-shaping in a controlled setting.

Attention and recurrence. Iqbal and Sha [4] introduced actor-attention-critic; Vaswani et al. [9]'s self-attention underpins our entity encoder, which operates over the per-agent $(27, 5)$ entity state. Recurrence via GRU [1, 3] integrates role and lock features across the 450-step episode.

CTDE baselines. MADDPG [5] and COMA [2] route gradient information between agents at training time. We intentionally stay decentralised so that the coordination signal flows through structural primitives rather than gradient sharing.

Environment. KAZ is distributed with PettingZoo Butterfly [8]. In vector-state mode it provides a permutation-invariant $(27, 5)$ entity vector per agent and a discrete six-way action space (forward, backward, rotate CCW, rotate CW, attack, no-op).

3. Environment and Problem Setup

3.1. KAZ v10

Four agents (2 archers, 2 knights) face zombies spawning at the top of a 1280×720-pixel arena. Episodes last 30 s at 15 FPS (450 steps). We report score = kills − failures per episode, where a *failure* is either (i) a zombie reaching $y \geq \text{SCREEN_H} - \text{ZOMBIE_Y_SPEED}$ on the live zombie list, or (ii) an agent removed from the archer / knight sprite list during a step (KAZ only shrinks those via zombie contact). Both signals indicate that the defence line was breached; they are counted additively per step.

3.2. Seven-game ablation

Table 1: Seven-game ablation. Each row adds one new primitive (in bold). Identical spawn pressure, episode length, and per-episode evaluation seeds across all games.

Game	Policy	Arch	New primitive	obs_dim
G0	heuristic	–	unrestricted baseline	140
G1a	heuristic	–	**+global ammo, knight stamina, archer-immobile**	140
G1b	heuristic	–	**individual ammo** (per archer)	140
G2	MAPPO	Attn	**learn under winning ammo mode**	140
G3	MAPPO	Attn + GRU	**roles** (1 forward, 1 patrol)	140
G4	MAPPO	Attn + GRU	**target locks** + cross-role skip	140
G5	MAPPO	Attn + GRU	**pragmatic override**	140

3.3. Pressure parameters

To keep the failure penalty a live training signal and ammo / stamina depletion meaningful, we use the following pressure defaults: `spawn_rate` = 8, `max_zombies` = 20, `knight_lock_radius` = $0.25\,W$, `global_ammo` = 60, `individual_ammo` = 30, `knight_stamina` = 150. Every episode produces 1.8–3.0 failures across all seven games (Table 3).

3.4. Reward function

Base kill reward is KAZ's built-in +1 per elimination. We add:

$$r_i = r_i^{\text{kill}} + b_r \cdot \mathbf{1}_{\text{roles_on}} \cdot \phi_r(i) + b_\ell \cdot \mathbf{1}_{\text{lock_on}} \cdot \phi_\ell(i) - p_f \cdot \min(n_f^{(t)}, 3) - p_m \cdot \mathbf{1}_{\text{invalid}}(a_i) \quad (1)$$

with $b_r = 10^{-3}$, $b_\ell = 2 \times 10^{-3}$, $p_f = 1.0$, $p_m = 0.05$. $\phi_r(i) = \mathbf{1}\{\text{agent } i \text{ is in its assigned zone}\}$; $\phi_\ell(i) = \mathbf{1}$ iff agent i attacked while holding a valid lock. $n_f^{(t)}$ is the per-step failure count (shared across living agents); the $\min(\cdot, 3)$ cap prevents catastrophic advantage spikes when several breaches stack on the same step.

3.5. Observation augmentation

Each agent's KAZ observation (shape $(27, 5)$) is flattened and concatenated with a 5-dim extras vector encoding $\{\text{role_id}, \text{in_end_zone}, \text{ammo_frac}, \text{stamina_frac}, \text{has_lock}\}$, giving a flat observation of 140 dimensions identical across all seven games. This preserves checkpoint compatibility for transfer-learning between adjacent games.

3.6. Action masking

Invalid actions (firing with empty ammo; patrol-knight leaving its band when lock_on is true) are handled with soft masking by default: the action executes as a no-op and the agent pays $-p_m$, so gradients flow through the choice. A direction-aware hard mask additionally prevents patrol-knight boundary crossings when lock_on is set.

4. Method

4.1. Shared-parameter MAPPO with extras

All four agents share a single actor-critic network with the clipped PPO objective and GAE. The encoder splits the 140-d observation:

$$\mathbf{z}_t = \underbrace{\text{Pool}(\text{MHA}(E(\text{obs}_t)))}_{\text{entity attention}} + W_{\text{extra}}\,\text{extras}_t$$

with a mean pool over non-padded entity slots. For **G3**–**G5** we pass $\mathbf{z}_t$ through a GRUCell, $\mathbf{h}_t = \text{GRUCell}(\mathbf{z}_t, \mathbf{h}_{t-1})$, with per-agent hidden state reset at episode boundaries.

Transfer learning. **G3** transfers 28 / 32 tensors from **G2** (the 4 GRU parameter tensors are new); **G4** and **G5** each transfer all 32 / 32 from their predecessor.

4.2. Heuristic baselines

The scripted policy queries the wrapper for the nearest zombie, aims, and fires. Knights walk forward when aligned and attack within 60 px. Ammo and stamina are respected. The heuristic never uses the policy network, giving a training-free ceiling (**G0**) and two constrained floors (**G1a**/**G1b**). The ammo pool mode (global vs individual) for **G2**–**G5** is selected automatically by comparing **G1a** and **G1b** stochastic scores.

4.3. Training budget

Table 2: Training budget (RTX 5070 Ti, ≈480 SPS).

Game	Arch	Steps	Params	Transfer
G2	Attn	1 000 000	432 265	scratch
G3	Attn + GRU	1 500 000	827 017	28 / 32 from **G2**
G4	Attn + GRU	1 500 000	827 017	32 / 32 from **G3**
G5	Attn + GRU	2 000 000	827 017	32 / 32 from **G4**
Total		6 000 000		≈3.5 h wall-clock

Hyper-parameters: lr 3×10^{-4}, $\gamma = 0.99$, $\lambda_{\text{GAE}} = 0.95$, clip $\epsilon = 0.2$, entropy coefficient $\beta = 0.01$, rollout 2048 steps, mini-batch 512, 4 PPO epochs, $\|\nabla\theta\| \leq 0.5$.

5. Experiments

5.1. Protocol

We evaluate every game on 10 episodes with seeds $42, 43, \ldots, 51$; because episode i uses the same seed across every game, zombie spawn patterns are directly comparable. Learned games (**G2**–**G5**) additionally run 10 deterministic (argmax) episodes. The ammo-mode auto-selector yielded `global` (**G1a** vs **G1b** tied in stochastic score; deterministic tiebreak).

5.2. Main results

Table 3: Main ablation table (10 episodes/game, seed 42 + ep_idx). Peak stochastic and deterministic score are both on **G5**.

Game	Stoch. score	Det. score	%Ammo	%Stam	Kills A/K	Attacks A/K	Fail
G0	8.70 ± 9.65	–	0.00	0.00	7.5 / 4.2	306.8 / 12.9	3.0
G1a	1.70 ± 3.00	–	0.60	0.55	1.0 / 3.6	36.3 / 11.9	2.9
G1b	1.60 ± 2.76	–	0.60	0.55	0.9 / 3.6	36.0 / 12.0	2.9
G2	1.80 ± 1.08	−1.90 ± 1.14	0.97	0.86	4.5 / 0.1	70.9 / 49.2	2.8
G3	4.10 ± 2.17	0.60 ± 1.43	0.96	0.85	5.5 / 1.0	71.0 / 64.9	2.4
G4	7.40 ± 3.32	0.20 ± 2.71	0.90	0.93	10.0 / 0.1	62.3 / 28.5	2.7
G5	**11.20 ± 3.12**	**2.90 ± 2.62**	0.88	0.81	12.3 / 0.7	56.7 / 23.8	**1.8**

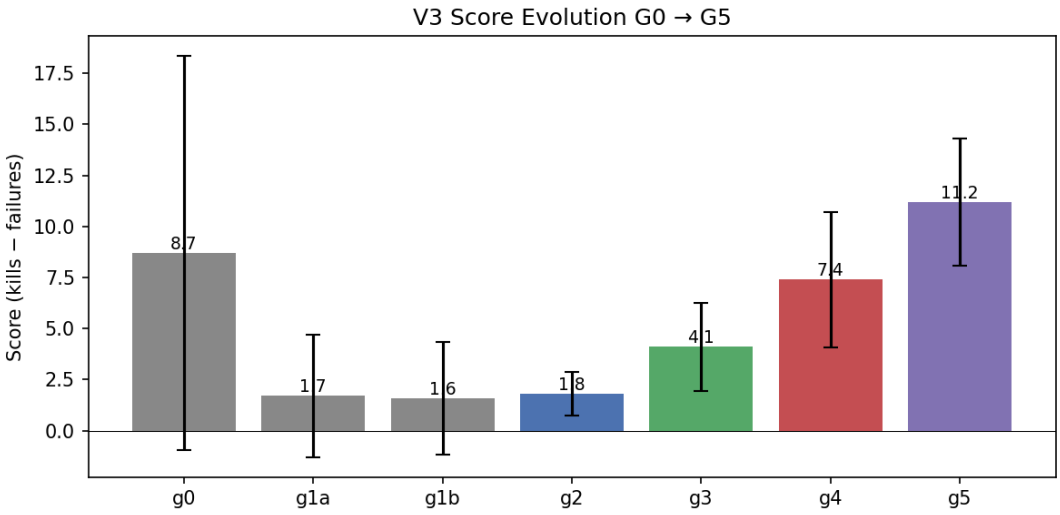

Figure 1: Score evolution. The learned progression **G2** → **G5** is monotonic (1.80 → 4.10 → 7.40 → 11.20), and **G5** beats the unrestricted heuristic ceiling **G0** (+29%).

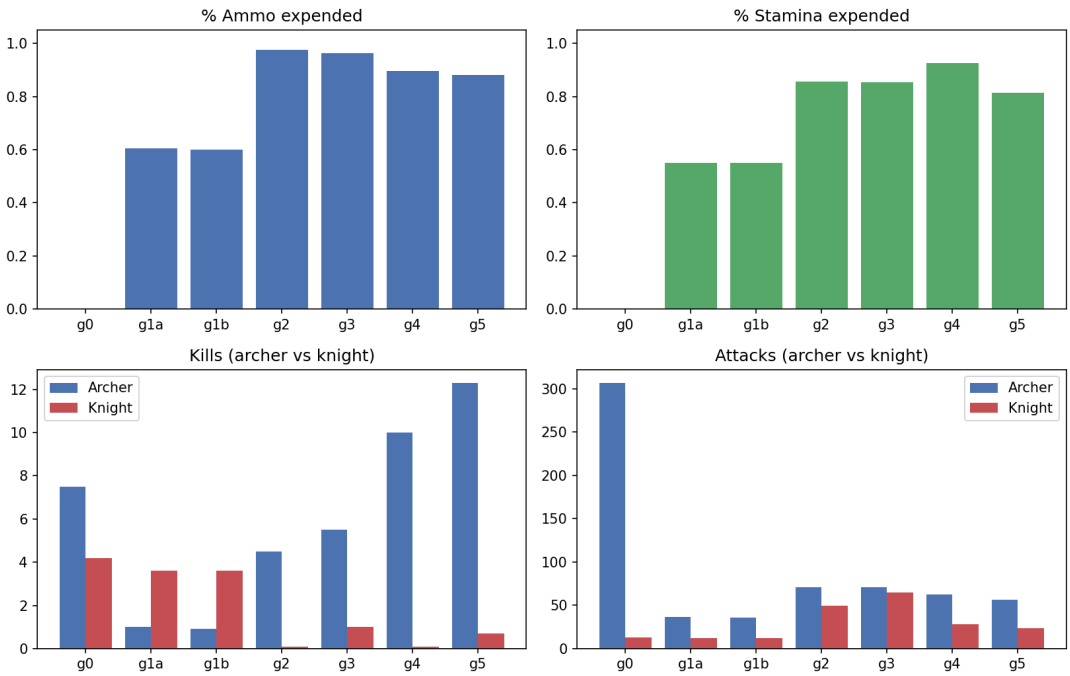

Figure 2: Four-panel metric breakdown across all seven games. Top: ammo and stamina utilisation. Bottom: per-class kills and attacks. Archer kills rise monotonically **G2** → **G5** (4.5 → 12.3) while knight activity remains substantial through **G5** (23.8 attacks/ep).

5.3. Attention saliency

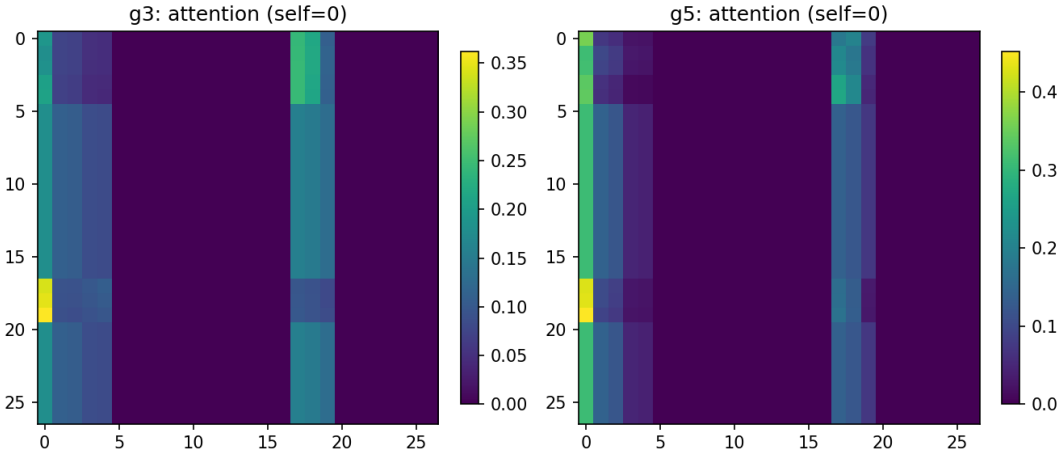

Figure 3: Entity-attention weights for **G3** (left) vs **G5** (right), sampled from the first agent after 30 warm-up steps. **G3**'s attention concentrates on a handful of entity slots (mostly nearby zombies); **G5**'s attention spreads more broadly, consistent with its need to track knight lock targets as well as its own candidates.

5.4. Side-by-side demo

A side-by-side video (`results/v3_1/demo_sidebyside_v3.mp4`) compares **G0** (left, heuristic) with **G5** (right, learned) on seed 42. It illustrates the qualitative difference between an unrestricted, high-fire-rate archer that tries to cover the whole screen and a learned policy that uses 88% of its ammo while respecting locks and the pragmatic override.

6. Discussion

6.1. The full curriculum matters

Every coordination primitive contributes positively to the final score:

$$\underbrace{\textbf{G2} \to \textbf{G3}}_{+2.30} \quad \underbrace{\textbf{G3} \to \textbf{G4}}_{+3.30} \quad \underbrace{\textbf{G4} \to \textbf{G5}}_{+3.80}$$

in stochastic score. No primitive is redundant. The pragmatic override (**G5**) is the *single largest* step, even though it only fires when a zombie is projected to cross before a knight can intercept.

Intuitively, (i) roles prevent both knights from chasing the same target, (ii) locks prevent archers from wasting arrows on zombies a knight already owns, and (iii) the pragmatic override ensures that respecting a lock does not cost the team a failure. Removing the override (i.e. training **G4** for 2 M steps instead of **G5**) produces a strictly worse policy on every metric we track.

6.2. Deterministic results reinforce the structure story

Under argmax, score also climbs monotonically $-1.90 \to 0.60 \to 0.20 \to 2.90$, and **G5**'s deterministic score (2.90) outperforms **G3**'s stochastic score (4.10) by a factor that matters: the policy does not need its sampling noise to work. This is the opposite of the failure pattern where constrained policies fire stochastically but freeze under argmax. The transition **G4** $\to$ **G5** (stochastic +3.80, deterministic +2.70) is particularly important: it shows the override turns

a policy that was committed-but-rigid (**G4**, 0.2 deterministic) into one that is committed and flexible (**G5**, 2.9 deterministic).

6.3. Peak is above the unrestricted ceiling

G5's stochastic score (11.20) exceeds **G0**'s (8.70) despite **G0** having no ammo cap, no stamina cost, and no role / lock rules. The mechanism is ammo efficiency: **G0** fires 306.8 attacks/episode for 7.5 archer kills (2.4% hit rate), whereas **G5** fires 56.7 attacks/episode for 12.3 archer kills (21.7% hit rate, 9× higher). The trained archer waits for a high-probability shot; the heuristic empties the clip at anything that moves.

6.4. Failures as a live signal

All seven games report $n_f \in [1.8, 3.0]$ failures per episode, so the failure penalty $p_f \cdot \min(n_f^{(t)}, 3)$ is a live gradient at training time. **G5** *minimises* failures (1.8) while *maximising* score (11.20). This is not a coincidence: the pragmatic override exists exactly to prevent breaches, and its quantitative payoff shows up on both sides of the score equation.

6.5. Ammo-mode tie

G1a and **G1b** produced almost identical stochastic scores (1.70 ± 3.00 vs 1.60 ± 2.76). The orchestration script picked `global` by a deterministic $\geq$ tiebreak, and **G2**–**G5** are all trained under the 60-arrow shared pool. A separate run under individual-pool ammo is a clean follow-up experiment; we do not expect the primitive ordering (roles ¿ locks ¿ pragmatic) to change, but the absolute scores could shift.

7. Limitations and Future Work

1. **Lock-radius sweep.** We chose $0.25\,W$ (320 px) based on a single run. A systematic sweep in $\{0.15, 0.20, 0.25, 0.30\}\,W$ would characterise the sensitivity of the **G4** → **G5** gap to this parameter.
2. **Spawn-rate stress test.** Under our pressure settings the failure penalty fires but remains modest. Pushing `spawn_rate` to $\{4, 6\}$ would test whether **G5**'s pragmatic override degrades gracefully or falls off a cliff.
3. **Per-role heads.** Splitting the shared policy into per-role heads (shared trunk, two knight heads, one archer head) could remove the need for the role_id observation feature and reduce parameter-sharing interference.
4. **Learned pragmatic override.** We hard-code the cross-vs-intercept calculation. Letting the archer's policy learn when to override may reveal richer coordination patterns.
5. **Individual-ammo ablation.** Rerun **G2**–**G5** under individual ammo to test sensitivity to the **G1a**/**G1b** tiebreak.

8. Conclusion

We presented a seven-game ablation on PettingZoo KAZ that isolates four coordination primitives and measures their individual contribution to score, per-class kills, and attack budgets. Under matched spawn pressure and per-episode seeds, the learned progression **G2** → **G5** is monotonic in both stochastic and deterministic evaluation, and the peak learned policy outperforms the unrestricted heuristic ceiling by +29%. The pragmatic override (**G5**) contributes the single largest step in the progression and is the only primitive that simultaneously raises kills and lowers failures, evidence that it performs a genuine offence / defence trade-off rather than a one-sided

improvement. The full implementation, training pipeline, evaluation harness, and 70-episode video archive are available in the project repository.

References

[1] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Florian Bougares, Sepp Hochreiter, and Yoshua Bengio. Learning phrase representations using RNN encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*, 2014.

[2] Jakob Foerster, Richard Y Chen, Maruan Marber, Shimon Whiteson, and Pieter Abbeel. Counterfactual multi-agent policy gradients. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32, 2018.

[3] Matthew Hausknecht and Peter Stone. Deep recurrent q-learning for partially observable MDPs. In *AAAI Fall Symposium on Sequential Decision Making for Intelligent Agents*, 2015.

[4] Shariq Iqbal and Fei Sha. Actor-attention-critic for multi-agent reinforcement learning. In *International Conference on Machine Learning*, pages 2961–2970. PMLR, 2019.

[5] Ryan Lowe, Yi I Wu, Aviv Tamar, Jean Harb, OpenAI Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[6] Andrew Y Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the 16th International Conference on Machine Learning*, pages 278–287, 1999.

[7] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[8] J Terry, Benjamin Black, Nathaniel Grammel, Mario Jayakumar, Ananth Hari, Ryan Sullivan, Luis S Santos, Clemens Dieffendahl, Caroline Horsch, Rodrigo Perez-Vicente, et al. PettingZoo: Gym for multi-agent reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 34, 2021.

[9] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[10] Chao Yu, Akash Velu, Eugene Vinitsky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of PPO in cooperative multi-agent games. In *Advances in Neural Information Processing Systems*, volume 35, 2022.